

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

I E. Sakthi priya (192520041) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Sakthi priya

Case study

QUESTIONS: Any 4

Total Marks:40

1. Online Hotel Reservation System

A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions.

Question:

Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

(10 Marks)

2. University Management System

A university intends to automate its operations including student admissions, course registration, examination management, and result processing. Different users such as students, faculty members, examination staff, and administrators interact with the system.

Question:

Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

(10 Marks)

3. Online Retail Shopping Platform

A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory.

Question:

Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.

(10 Marks)

4. Smart Waste Management System

A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application.

Question:

Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why.

(10 Marks)

5. Smart Healthcare Management System

A multi-specialty hospital plans to develop a Smart Healthcare Management System where patients can book appointments, consult doctors, access medical records, and make online payments. Doctors update patient diagnoses and prescriptions, while hospital administrators manage departments, staff, and billing information.

Question:

Analyze the system and explain how **object-oriented principles (encapsulation, inheritance, and polymorphism)** can be applied. Also, recommend a suitable **SDLC model** for developing this healthcare system and justify your choice.

(10 Marks)

6. Smart Library Management System

A university library intends to automate its operations, allowing students and faculty members to search books, borrow and return books, reserve titles online, and pay overdue fines. Librarians manage book inventories, memberships, and circulation records, while administrators generate reports and maintain the system.

Question:

Design a structured solution using **object-oriented concepts** for this system. Identify the major classes, attributes, methods, and relationships, and recommend an appropriate **SDLC model** for efficient development with suitable justification.

(10 Marks)

RUBRICS FOR CALCULATION:

Case study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1) Online Hotel Reservation System

1. Application of Object-Oriented Principles

a) Encapsulation

Encapsulation means binding data and methods together inside a class and restricting direct access to the data.

In the hotel reservation system:

- A Customer class contains customer details such as customer ID, name, email, and phone number.
- A Room class contains room number, room type, price, and availability.
- A Booking class contains booking ID, check-in date, check-out date, and booking status.
- Data can be kept private and accessed through methods such as `getRoomDetails()`, `updateAvailability()`, and `makeBooking()`.
- This protects important information such as customer and payment details from unauthorized access.

b) Inheritance

Inheritance allows a new class to reuse the properties and methods of an existing class.

For example:

- User can be the parent class.
- Customer, HotelManager, and Administrator can inherit from the User class.
- Common attributes such as user ID, name, email, and password can be defined in User.
- Each child class can have its own specific functions.
- This reduces code duplication and improves code reusability.

Example:

User

↓

Customer | HotelManager | Administrator

c) Polymorphism

Polymorphism means the same method can perform different actions depending on the object.

For example:

- A `makePayment()` method can be defined in a Payment class.
- Different payment classes such as `CreditCardPayment`, `UPIPayment`, and `NetBankingPayment` can implement the method differently.

- When makePayment() is called, the appropriate payment method is executed based on the selected payment type.
- Similarly, a generateReport() method can produce different reports for administrators and hotel managers.

2. Main Classes

Class	Main Responsibility
User	Stores common user information
Customer	Searches hotels, books rooms, makes payments, and provides reviews
HotelManager	Updates room availability and pricing
Administrator	Manages users and transactions
Hotel	Stores hotel details and manages rooms
Room	Stores room type, price, and availability
Booking	Manages room reservations
Payment	Handles payment transactions
Review	Stores customer reviews and ratings

3. Suitable SDLC Model – Agile Model

The **Agile SDLC model** is suitable for the Online Hotel Reservation System.

Justification

- The system has many features such as hotel search, room booking, payment, reviews, and user management.
- Requirements may change based on customer and hotel manager feedback.
- Agile develops the system in **small iterations or sprints**.
- Features such as hotel search and booking can be developed first, followed by payment, reviews, and administration.
- Testing is performed continuously during development.
- Customers and hotel managers can provide feedback after each iteration.
- Errors and changes can be identified and corrected early.
- Agile helps deliver a working system quickly and improves the system continuously.

4. Agile Development Process

Requirement Analysis → Sprint Planning → Design & Development → Testing → Customer Feedback → Improvements → Next Sprint

5. Conclusion

The Online Hotel Reservation System can effectively use encapsulation to protect data, inheritance to reuse common user properties, and polymorphism to support different implementations such as payment methods. The Agile SDLC model is suitable because it supports changing requirements, continuous testing, frequent feedback, and incremental development.

3) Online Retail Shopping Platform

1. Modular Software Design Using Object-Oriented Techniques

Modular software design means **dividing the system into separate modules**, where each module performs a specific function. In an online retail shopping platform, object-oriented techniques can be used to create independent and reusable modules.

a) Product Module

- Manages product details such as product ID, name, price, category, and quantity.
- A Product class can be used to represent products.
- Vendors can add, update, or remove products from the catalog.

b) Customer Module

- Manages customer registration, login, profile, and product browsing.
- A Customer class can contain customer details and methods such as `browseProduct()` and `updateProfile()`.

c) Order Module

- Handles placing and managing customer orders.
- An Order class can contain order ID, date, status, and ordered products.
- Methods such as `placeOrder()` and `cancelOrder()` can be included.

d) Payment Module

- Handles online payment transactions.
- A Payment class can manage payment details and status.
- Different payment methods such as UPI, card, and net banking can be implemented as separate classes.

e) Delivery Module

- Manages shipping and delivery tracking.

- A Delivery class can store delivery address, tracking ID, and delivery status.

f) Inventory Module

- Maintains product stock information.
- An Inventory class can update stock when products are purchased or restocked.

g) Administrator Module

- Allows administrators to monitor transactions, products, orders, and inventory.
- An Administrator class can provide functions for managing the system.

2. Object-Oriented Techniques for Modularity

- **Encapsulation:** Keeps the data and methods of each module together and protects internal data.
- **Abstraction:** Hides unnecessary implementation details and exposes only required operations.
- **Inheritance:** Allows common properties and methods to be reused between related classes.
- **Polymorphism:** Allows different classes to implement the same operation in different ways.
- **Reusability:** Individual classes and modules can be reused in different parts of the system.
- **Low Coupling:** Modules should depend on each other as little as possible, making changes easier.

3. Evaluation of SDLC Models

Waterfall Model

- Follows a fixed sequence of development stages.
- Easy to understand and manage.
- Not suitable when requirements may change frequently.
- Changes are difficult to accommodate after development begins.

Spiral Model

- Combines iterative development with risk analysis.
- Suitable for large and high-risk systems.
- Requires more time, cost, and management effort.
- May be unnecessarily complex for a normal e-commerce platform.

Prototype Model

- A working prototype is created early.

- Helps understand customer requirements and user interface needs.
- Useful when requirements are unclear.
- Continuous development may still be needed after the prototype.

Agile Model

- Develops the system in small iterations or sprints.
- Allows frequent changes based on customer feedback.
- Supports continuous testing and improvement.
- Features can be developed separately, such as product browsing, ordering, payment, and delivery tracking.

4. Most Appropriate SDLC Model – Agile Model

The **Agile SDLC model** is the most appropriate for the Online Retail Shopping Platform.

Justification

- E-commerce requirements can change frequently based on customer and vendor needs.
- The system contains many independent features that can be developed in separate sprints.
- Customer feedback can be incorporated continuously.
- Testing can be performed during every development iteration.
- New features such as discount systems, new payment methods, and delivery options can be added easily.
- Problems can be identified and fixed early.
- Agile provides faster delivery of working features.
-

5. Agile Development Process

Requirement Analysis → Sprint Planning → Design & Development → Testing → Customer Feedback → Improvements → Next Sprint

6. Conclusion

Object-oriented techniques allow the Online Retail Shopping Platform to be divided into independent and reusable modules such as Product, Customer, Order, Payment, Delivery, Inventory, and Administration. Among the different SDLC models, Agile is the most suitable because it supports changing requirements, continuous testing, customer feedback, and incremental development.

4) Smart Waste Management System

1. Object-Oriented System Development

The Smart Waste Management System can be developed using object-oriented principles by dividing the system into different classes and modules.

a) Bin Module

- Represents each smart waste bin.
- Stores bin ID, location, waste level, and status.
- Methods can include `checkWasteLevel()` and `sendNotification()`.

b) Sensor Module

- Monitors the waste level inside the bin.
- Sends real-time data to the central system.
- A Sensor class can contain sensor ID and reading information.

c) Collection Vehicle Module

- Represents vehicles used for waste collection.
- Stores vehicle ID, location, and assigned bins.
- Methods can include `receiveAlert()` and `collectWaste()`.

d) Citizen Module

- Allows citizens to use the mobile application.
- Citizens can report issues such as overflowing bins or uncollected waste.
- A Citizen class can contain citizen details and reporting methods.

e) Notification Module

- Sends alerts to collection vehicles or municipal staff when bins are full.
- Can also send updates about reported waste issues.

f) Waste Management System

- Collects and processes data from smart bins.
- Monitors bin status and assigns collection vehicles.
- Maintains waste collection records.

2. Application of Object-Oriented Principles

Encapsulation

- Each class keeps its data and methods together.

- For example, the SmartBin class can protect its waste-level data and provide methods to access or update it.
- This prevents unauthorized modification of sensor and bin information.

Abstraction

- Complex sensor operations are hidden from users.
- The system only displays useful information such as Bin Full, Bin Half Full, or Bin Empty.

Inheritance

- A general User class can be created.
- Citizen and MunicipalStaff can inherit common properties from the User class.
- This improves code reuse.

Polymorphism

- Different types of sensors can use the same readData() method.
- For example, an ultrasonic sensor and an IoT level sensor can implement readData() differently.

3. Object-Oriented System Development Steps

1. Identify Objects – Smart Bin, Sensor, Collection Vehicle, Citizen, Notification, and Waste Management System.
2. Identify Classes – Create classes based on the identified objects.
3. Define Attributes and Methods – Assign data and operations to each class.
4. Define Relationships – Connect sensors with bins, bins with collection vehicles, and citizens with issue reports.
5. Apply OOP Principles – Use encapsulation, abstraction, inheritance, and polymorphism.
6. Develop and Test Modules – Implement and test each module independently.
7. Integrate the System – Connect IoT sensors, mobile application, and central management system.

4. Life Cycle Model – Agile Model

The Agile SDLC model is best suited for the Smart Waste Management System.

Reasons

- The system receives real-time data continuously from IoT-enabled sensors.
- Sensor and IoT integration may require frequent testing and modifications.

- Requirements may change based on municipal needs and citizen feedback.
- The system can be developed in small modules such as smart bins, notifications, vehicle management, and mobile reporting.
- Continuous testing helps identify sensor and communication problems early.
- New features can be added easily in future iterations.
- Feedback from municipal staff and citizens can be incorporated into later sprints.

Agile Development Process

Requirement Analysis → Sprint Planning → IoT & System Development → Testing → Deployment → User Feedback → Improvements → Next Sprint

5. Conclusion

The Smart Waste Management System can be effectively developed using object-oriented principles by creating separate classes for smart bins, sensors, collection vehicles, citizens, notifications, and waste management. Agile is the most suitable life cycle model because it supports real-time sensor integration, continuous testing, changing requirements, and frequent user feedback.

5) Smart Healthcare Management System

1. Application of Object-Oriented Principles

a) Encapsulation

Encapsulation means combining data and methods within a class and restricting direct access to the data.

In the healthcare system:

- A Patient class stores patient ID, name, contact details, and medical information.
- A Doctor class stores doctor ID, name, specialization, and availability.
- An Appointment class stores appointment date, time, and status.
- A MedicalRecord class stores diagnosis, prescriptions, and medical history.
- Sensitive information can be kept private and accessed through methods such as `viewMedicalRecord()`, `updateDiagnosis()`, and `updatePrescription()`.
- This helps protect confidential patient information.

b) Inheritance

Inheritance allows a class to inherit common properties and methods from another class.

For example:

User



Patient | Doctor | Administrator

- User can contain common attributes such as user ID, name, email, and password.
- Patient, Doctor, and Administrator inherit these common properties.
- Each subclass can have its own specific methods.
- This improves code reuse and reduces duplication.

c) Polymorphism

Polymorphism allows the same method to perform different operations depending on the object.

For example:

- A makePayment() method can be implemented differently for UPIPayment, CardPayment, and NetBankingPayment.
- A generateReport() method can generate different reports for doctors and administrators.
- A viewMedicalRecord() operation can provide different levels of access depending on whether the user is a patient or doctor.

2. Main Classes

Class	Responsibility
User	Stores common user information
Patient	Books appointments and accesses medical records
Doctor	Consults patients and updates diagnosis and prescriptions
Administrator	Manages departments, staff, and billing
Appointment	Manages appointment details
MedicalRecord	Stores diagnosis, prescriptions, and medical history
Payment	Handles online payments
Department	Manages hospital departments
Billing	Manages billing information

3. Suitable SDLC Model – Agile Model

The Agile SDLC model is suitable for developing the Smart Healthcare Management System.

Justification

- The system contains many modules such as appointments, medical records, consultation, payments, and billing.
- These modules can be developed and tested in separate sprints.
- Healthcare requirements may change based on doctors', patients', and hospital administrators' needs.
- Continuous testing helps identify errors early.
- Feedback from doctors and hospital staff can be incorporated during development.
- New features can be added without redesigning the entire system.
- Agile allows frequent releases and continuous improvement.

4. Agile Development Process

Requirement Analysis → Sprint Planning → Design & Development → Testing →
Deployment → User Feedback → Improvements → Next Sprint

5. Conclusion

The Smart Healthcare Management System can use encapsulation to protect sensitive patient data, inheritance to reuse common properties among patients, doctors, and administrators, and polymorphism to support different payment methods and user-specific operations. The Agile SDLC model is suitable because it supports modular development, continuous testing, changing requirements, and regular feedback from healthcare users.